

Deploying a RAG Python File Using Docker + Kubernetes in Azure

This guide explains how to deploy a Retrieval-Augmented Generation (RAG) Python application using Docker and Kubernetes on Microsoft Azure.

Step 1: Containerize Your Python File (Docker)

1. Create a Dockerfile in the same directory as your RAG Python script:

```
FROM python:3.10-slim
WORKDIR /app
COPY . /app
RUN pip install --no-cache-dir -r requirements.txt
EXPOSE 8000
CMD ["python", "your_rag_script.py"]
```

2. Create a requirements.txt file:

```
langchain
langchain-core
langchain-community
langchain-openai
beautifulsoup4
chromadb
tiktoken
requests
```

3. Build and tag the Docker image:

```
docker build -t rag-app .
```

Step 2: Push Docker Image to Azure Container Registry (ACR)

1. Login to Azure:

az login

2. Create Resource Group:

az group create --name rag-rg --location eastus

3. Create Azure Container Registry:

az acr create --resource-group rag-rg --name myragregistry --sku Basic

4. Push Docker image:

az acr login --name myragregistry

docker tag rag-app myragregistry.azurecr.io/rag-app:v1

docker push myragregistry.azurecr.io/rag-app:v1

Step 3: Deploy to Kubernetes (Azure AKS)

1. Create AKS Cluster:

az aks create --resource-group rag-rg --name rag-aks --node-count 1 --enable-addons monitoring
--generate-ssh-keys

2. Get AKS credentials:

az aks get-credentials --resource-group rag-rg --name rag-aks

3. Create Kubernetes deployment YAML (rag-deploy.yaml):

apiVersion: apps/v1

kind: Deployment

metadata:

name: rag-app-deployment

spec:

replicas: 1

selector:

matchLabels:

app: rag-app

template:

metadata:

labels:

app: rag-app

spec:

containers:

- name: rag-app

image: myragregistry.azurecr.io/rag-app:v1

ports:

- containerPort: 8000

imagePullPolicy: Always

imagePullSecrets:

- name: acr-auth

apiVersion: v1

kind: Service

metadata:

name: rag-app-service

spec:

type: LoadBalancer

ports:

- port: 80

targetPort: 8000

selector:

app: rag-app

4. Create image pull secret:

ACR_USERNAME=\$(az acr credential show -n myragregistry --query "username" -o tsv)

ACR_PASSWORD=\$(az acr credential show -n myragregistry --query "passwords[0].value" -o tsv)

```
kubectl create secret docker-registry acr-auth --docker-server=myragregistry.azurecr.io  
--docker-username=$ACR_USERNAME --docker-password=$ACR_PASSWORD
```

--docker-email=you@example.com

5. Apply deployment:

```
kubectl apply -f rag-deploy.yaml
```

6. Get External IP:

```
kubectl get service rag-app-service
```

Access your app at: <http://<EXTERNAL-IP>/>

Optional Enhancements:

- Use FastAPI or Flask to expose endpoints
- Use Helm charts
- Add Ingress controller + TLS for HTTPS